Universidad Tecnológica de la Mixteca

Ingeniería en Mecatrónica

Circuitos Digitales Reconfigurables

Práctica 1: Decodificador Hexadecimal con Control de Polaridad en CUPL

Docente: Dr. Enrique Guzmán Ramírez

Integrantes: Agudo Hernández Alondra
Giovanni Alvarado Herrera
Ricardo Ignacio Mendoza Peralta
Emmanuel Santiago García

Grupo: 614-A

Fecha: 20 de febrero de 2026

Huajuapan de León, Oaxaca, México

Índice de Contenido

1. Resumen **1**

2. Introducción **1**

3. Desarrollo y Justificación del Código **1**

3.1. Definición de Entradas y Agrupamiento de Datos 1

3.2. Lógica de Decodificación y Control de Polaridad 2

3.3. Ecuaciones de los Segmentos Restantes 2

4. Resultados y Funcionamiento **2**

5. Conclusiones **3**

6. Evidencia Fotográfica y Simulaciones **3**

7. Referencias **7**

1. Resumen

Este reporte detalla el diseño y la implementación de un decodificador de 4 bits (hexadecimal) para un display de siete segmentos utilizando una GAL22V10. La innovación de este diseño radica en la inclusión de un pin de control denominado `modo`, el cual permite alternar la lógica de salida mediante el operador XOR. Esto otorga al sistema la versatilidad de operar tanto con displays de Ánodo Común como de Cátodo Común sin necesidad de modificar el hardware físico, optimizando así el uso de dispositivos lógicos programables (PLD).

2. Introducción

La visualización de datos numéricos en sistemas digitales se realiza predominantemente mediante displays de siete segmentos. Tradicionalmente, un decodificador comercial como el 74LS47 está limitado a una sola polaridad y a caracteres decimales. Mediante el uso de lenguaje CUPL y lógica reconfigurable, es posible diseñar un decodificador a medida que no solo soporte el alfabeto hexadecimal completo (0-F), sino que también implemente funciones de control de fase lógica. En esta práctica, se exploran las directivas de conjunto (`field`) y el mapeo de tablas de verdad mediante valores hexadecimales para simplificar las ecuaciones booleanas de salida.

3. Desarrollo y Justificación del Código

El diseño se fundamenta en la manipulación de una palabra de entrada de 4 bits y una señal de control de modo. A continuación, se desglosa la lógica implementada:

3.1. Definición de Entradas y Agrupamiento de Datos

Para facilitar la lectura del código, se agrupan las señales de entrada en un campo vectorial:

```
PIN [2..5] = [w, x, y, z]; /* Entradas BCD/Hex */
PIN 6 = modo;              /* 0 = Catodo Comun, 1 = Anodo Comun */
field s = [x, y, z, w];    /* Definicion del vector de estado */
```

Justificación: El uso de `field` permite referenciar los 4 switches como una sola variable `s`. Esto es crucial para emplear la sintaxis de comparación de estados (`s:'h'[...]`), lo que reduce significativamente la probabilidad de errores en las tablas de verdad manuales.

3.2. Lógica de Decodificación y Control de Polaridad

La lógica de los segmentos (a al g) se define mediante la identificación de los estados en los que cada segmento debe estar **apagado** (asumiendo lógica de cátodo común por defecto).

```
¬a = modo ⊕ (s:'h'[1, 4, b, d]);
¬b = modo ⊕ (s:'h'[5, 6, b, c, e, f]);
¬c = modo ⊕ (s:'h'[2, c, e, f]);
```

Justificación técnica:

- **El operador NOT (!):** Se utiliza para definir la ecuación en términos de cuándo el segmento está inactivo.
- **Comparación Hexadecimal ('h'):** En lugar de escribir minitérminos como `(!x & !y & !z & w)`, se listan los valores hexadecimales donde el segmento se apaga. Por ejemplo, el segmento `a` se apaga en los números $1, 4, B$ y D.
- **Control XOR ($):** Esta es la parte fundamental del diseño. La operación $A \oplus B$ actúa como un inversor programable. Si `modo = 0`, la salida mantiene su valor original. Si `modo = 1`, la salida se invierte totalmente, permitiendo que el mismo chip controle un display de Ánodo Común.

3.3. Ecuaciones de los Segmentos Restantes

Siguiendo la misma metodología, se completan los segmentos para formar los caracteres hexadecimales:

```
¬d = modo ⊕ (s:'h'[1, 4, 7, a, f]);
¬e = modo ⊕ (s:'h'[1, 3, 4, 5, 7, 9]);
¬f = modo ⊕ (s:'h'[1, 2, 3, 7, d]);
¬g = modo ⊕ (s:'h'[0, 1, 7, c]);
```

4. Resultados y Funcionamiento

Durante las pruebas experimentales, se observó que el sistema decodifica correctamente los 16 estados del campo `s`.

Cuadro 1: Prueba de Modo de Operación

Entrada Modo	**Lógica Resultante**	**Tipo de Display**	**Salida Activa**
0	Directa	Cátodo Común	Nivel Alto (1)
1	Invertida	Ánodo Común	Nivel Bajo (0)

Por ejemplo, al ingresar el valor `'h'A` (1010 en binario), los segmentos configurados en las ecuaciones muestran la letra A en el display. Si el usuario cambia el switch de `modo`, el brillo de los segmentos se invierte instantáneamente, validando la ecuación de control $Salida = Modo \oplus Logica$.

5. Conclusiones

La implementación de este decodificador en la GAL22V10 demuestra la superioridad de los PLDs sobre la lógica de función fija. Mediante el uso de directivas avanzadas de CUPL como `field` y operadores de conjunto, se logró un diseño compacto y altamente funcional. La inclusión del control de polaridad mediante la compuerta XOR es una técnica eficiente que dota al circuito de flexibilidad industrial, permitiendo su adaptación a diferentes periféricos de salida sin cambios en el cableado.

6. Evidencia Fotográfica y Simulaciones

En esta sección se presentan las evidencias del correcto funcionamiento del diseño.

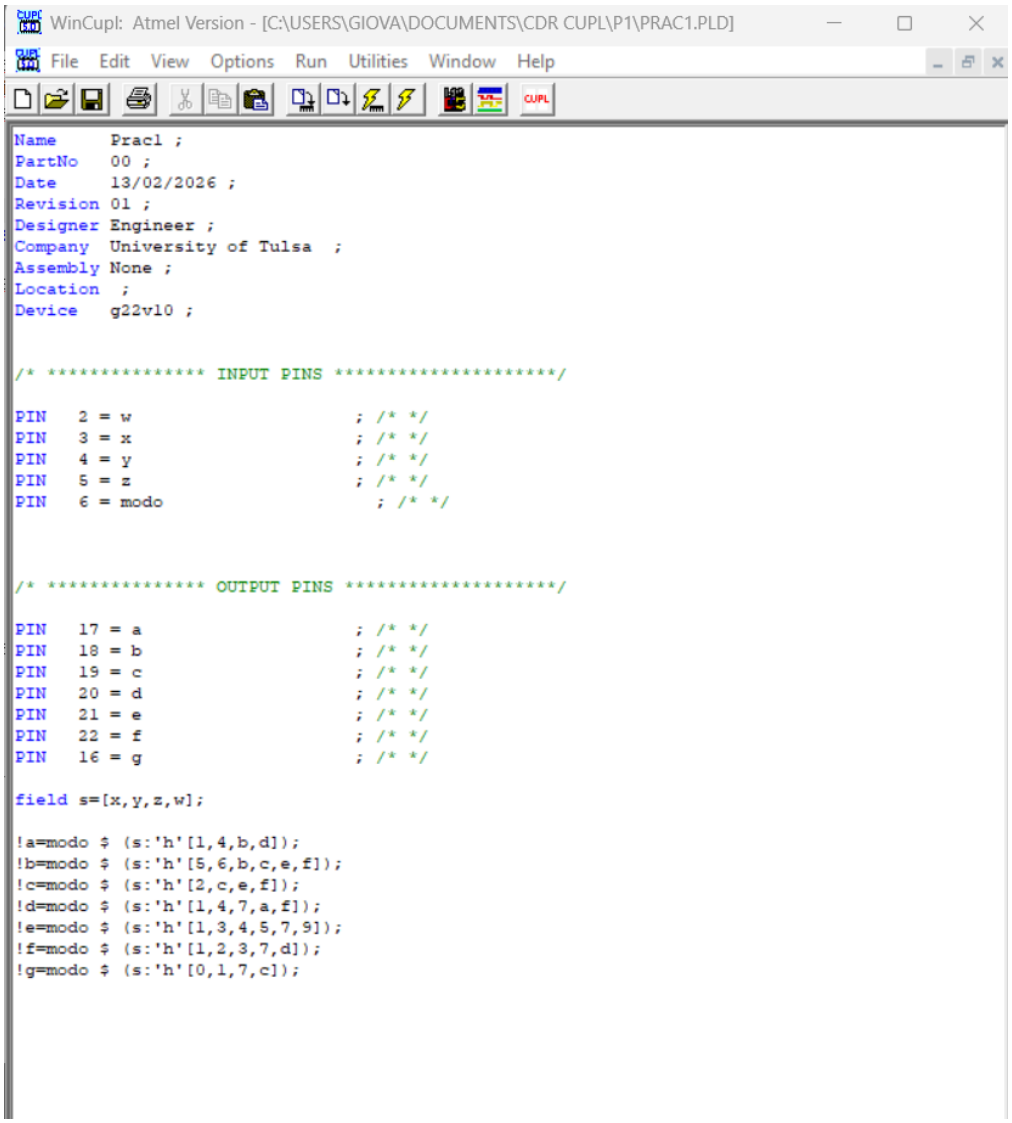


Figura 1: Captura de pantalla del codigo completo implementado.

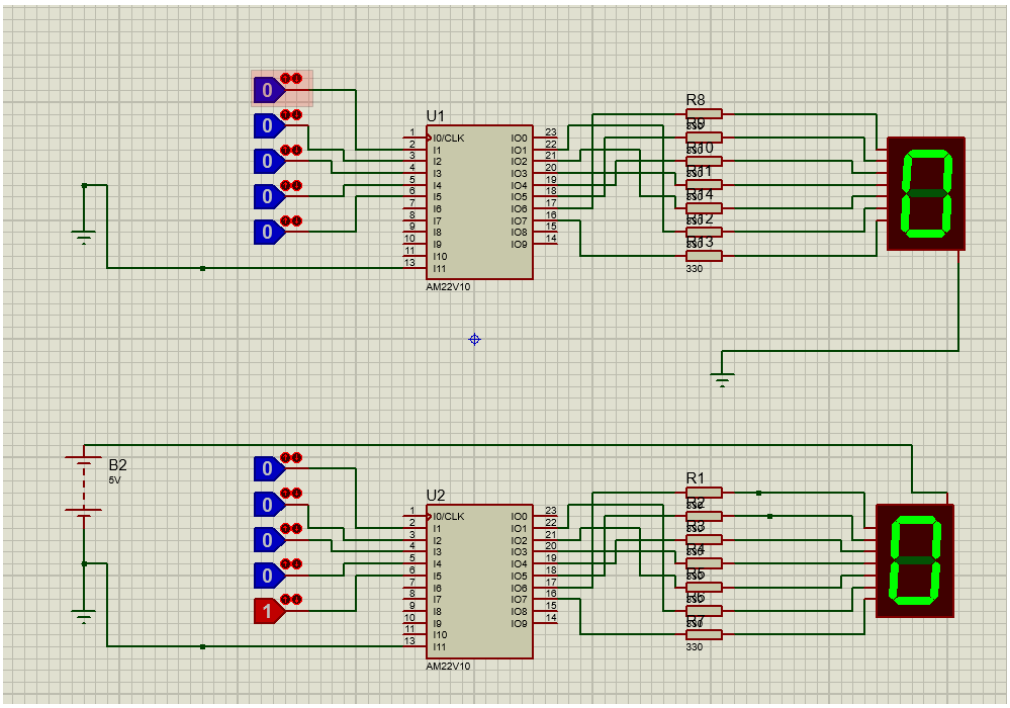


Figura 2: Captura de pantalla de la simulacion en Proteus.

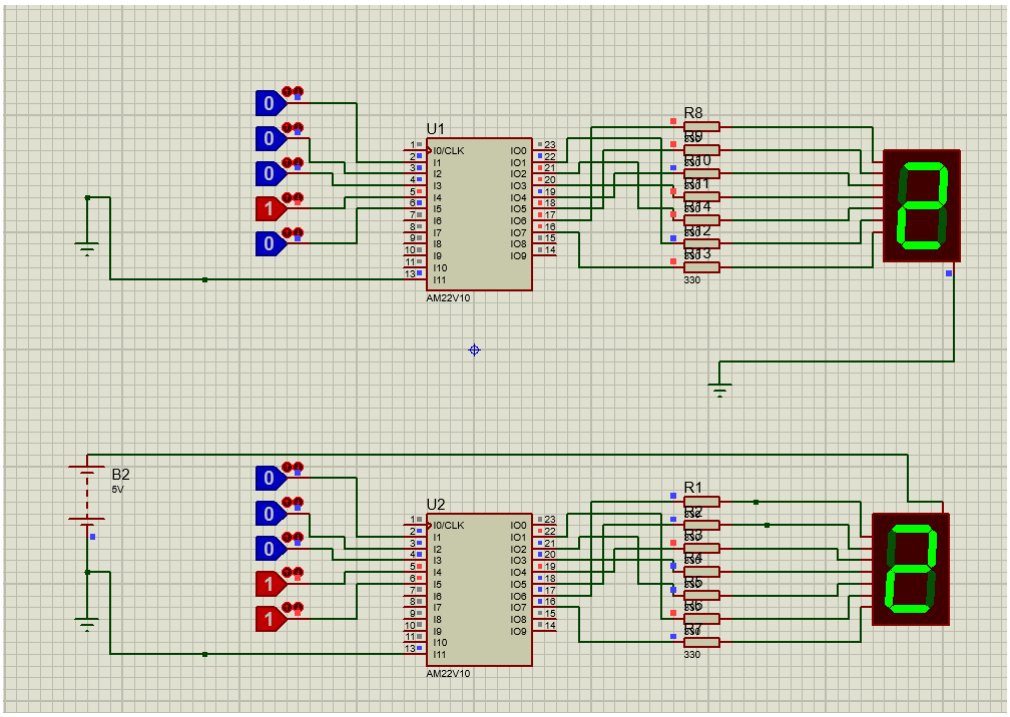


Figura 3: Captura de pantalla de la simulacion en Proteus.

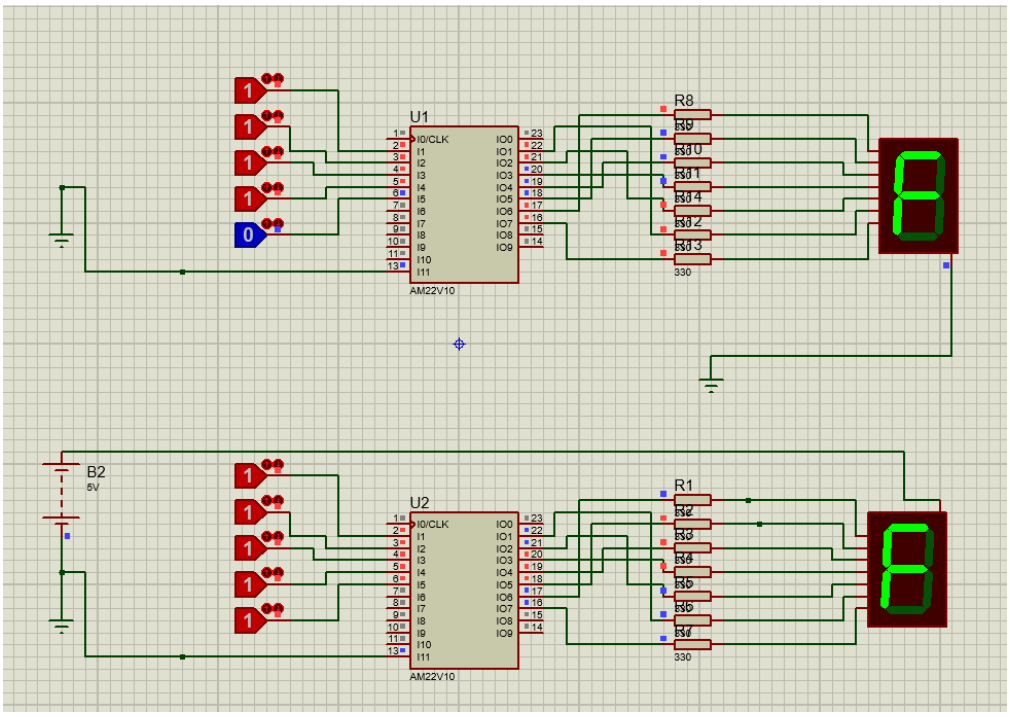
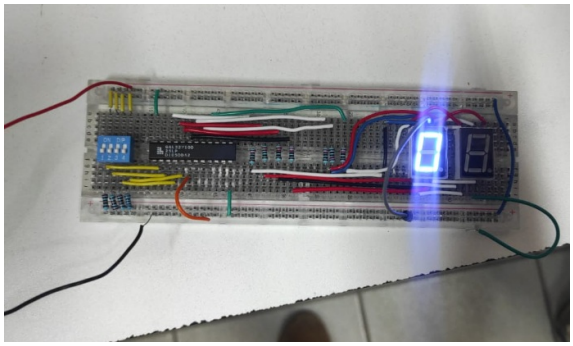
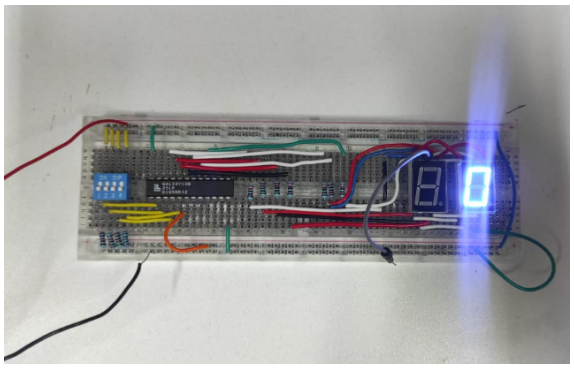


Figura 4: Captura de pantalla de la simulacion en Proteus.

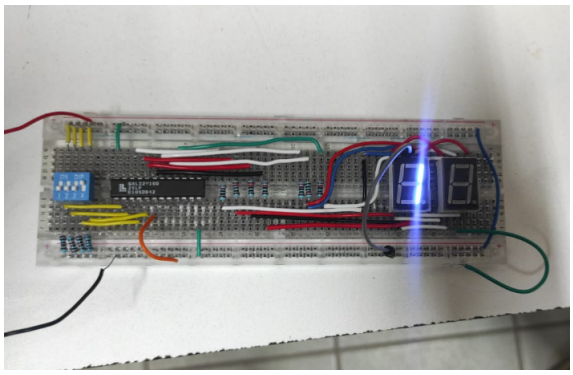


(a) Evidencia en modo Cátodo Común.

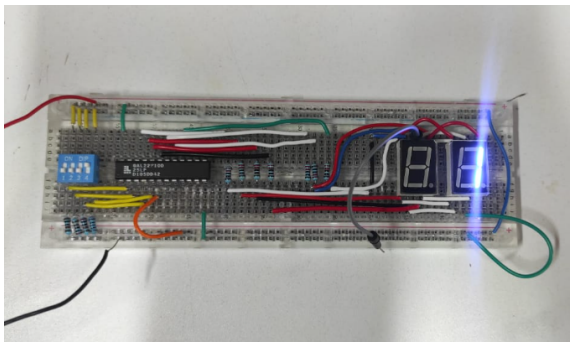


(b) Evidencia en modo Ánodo Común.

Figura 5: Pruebas de funcionamiento del decodificador con control de polaridad.

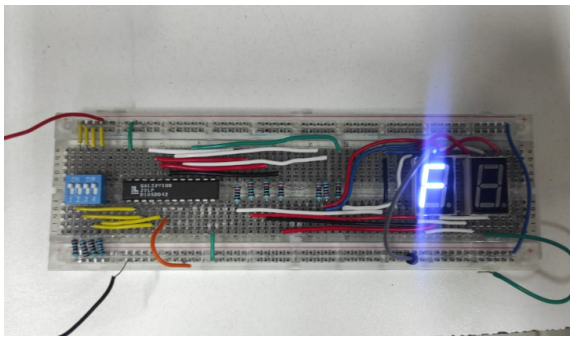


(a) Evidencia en modo Cátodo Común.

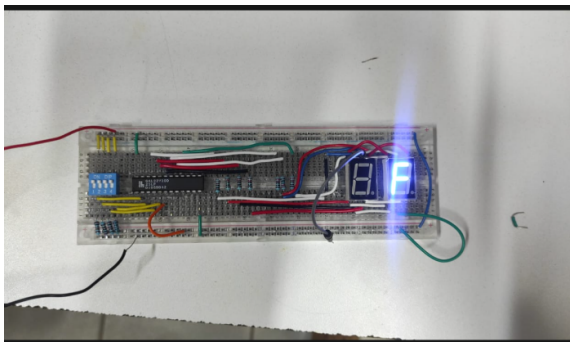


(b) Evidencia en modo Ánodo Común.

Figura 6: Pruebas de funcionamiento del decodificador con control de polaridad.



(a) Evidencia en modo Cátodo Común.



(b) Evidencia en modo Ánodo Común.

Figura 7: Pruebas de funcionamiento del decodificador con control de polaridad.

7. Referencias

- Lattice Semiconductor. *GAL22V10 Data Sheet*.
- Tocci, R. J., Widmer, N. S., & Moss, G. L. (2017). *Sistemas Digitales: Principios y Aplicaciones*. Pearson.